Instructions:
1. Run task_2.py to generate the tensor database if not already generated.
2. Press Run All (or restart kernel and run all cells).
3. You will be prompted to provide an image ID and a k value.

Task 3: Implement a program which, given an image ID and a value “k”, returns and visualizes the most similar k images based on each of the visual model –you will select the appropriate distance/similarity measure for each feature model.
For each match, also list the corresponding distance/similarity score.

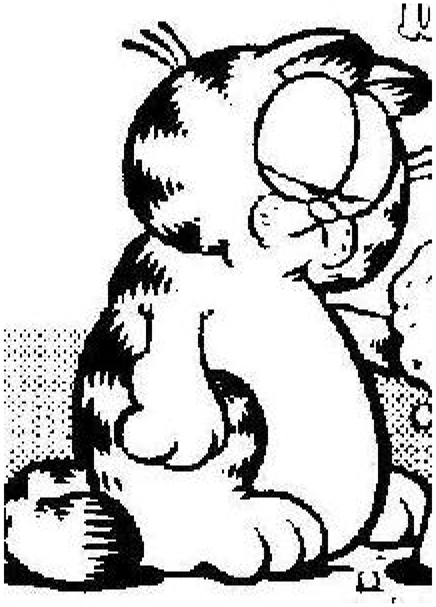
In [ ]:
```
IMG_ID = int(input("Provide an Image ID in the Caltech101 dataset in range [0, 8676]."))

K = int(input("Enter K, the number of similar images K to find for each feature model."))
```

In [ ]:
```
# Initialize dataset, downloads into <git_root>/Code/caltech101 if not present. Untracked by git.

import os
from torchvision.datasets import Caltech101

dataset = Caltech101(root=os.path.abspath(""), download=True)
```

```
Files already downloaded and verified
```

In [ ]:
```
# Find the k most similar images for a given Image ID for each feature model.

image, category = dataset[IMG_ID][0], dataset.categories[dataset[IMG_ID][1]]

if image.mode != 'RGB':
    print("WARNING: The provided image does not have RGB channels, the image will be converted to RGB.")
    image = image.convert('RGB')

print("Input image:\nImage ID: ", IMG_ID)
display(image)
print("Category:", category)
```

```
WARNING: The provided image does not have RGB channels, the image will be converted to RGB.
Input image:
Image ID:  5122
```



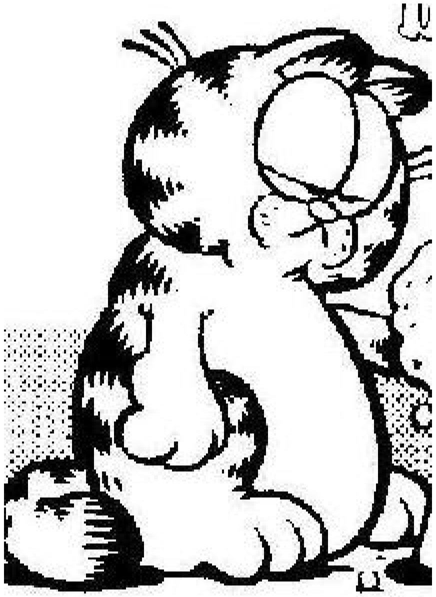
Category: garfield

In [ ]:
```
# Resize to 300x100 and partition to grid.

image300x100 = image.resize((300, 100))

from utils.utilities import partition_to_grid

grid = partition_to_grid(image300x100, num_rows=10, num_cols=10, hor_pixels=30, ver_pixels=10)
```

In [ ]:
```
# Color moments (CM10x10).

# Find K most similar images by color moments.

from feature_extractors.color_moments import get_color_vector

color_vector = get_color_vector(grid)

# Load color tensor database into memory.

from utils.utilities import tensor_loader

color_tensor_dict = tensor_loader('color_tensors.pt', os.path.abspath(""))
```

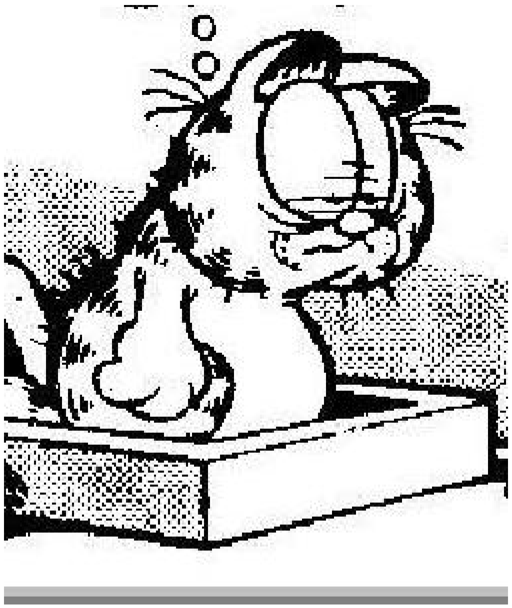
```python
from utils.utilities import top_k_ranker

# Manhattan distance (City-block distance): This distance can be imagined as the length needed
# to move between two points in a grid where you can only move up, down, left or right. This
# measure is sensitive to differences along each dimension of the feature vectors, capturing
# the total absolute difference between corresponding moments, which makes it great for
# assessing differences in color distribution in terms of absolute deviations. Since it
# does not square the differences like Euclidean distance, it is less susceptible outliers
# when many dimensions differ.

# We use Earth Mover's Distance (Wasserstein distance) as color moments can be thought
# of as probability distributions over color space. EMD is capable of comparing distributions
# with different shapes and scales, color moments consider the spatial arrangement of colors,
# not just the frequency of colors. EMD takes this spatial information into account by
# considering how color probability mass is transported from one distribution to another.

from scipy.spatial.distance import cityblock

print("Similar images by Color Moments using distance: Manhattan (cityblock)")

top_k_by_color = top_k_ranker(K, color_vector, color_tensor_dict, cityblock)


for i in range(len(top_k_by_color)):

    id = top_k_by_color[i][1]
    distance = top_k_by_color[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])
```

```
Similar images by Color Moments using distance: Manhattan (cityblock)
1.      Image ID:  5122                 Distance:  0.0
```

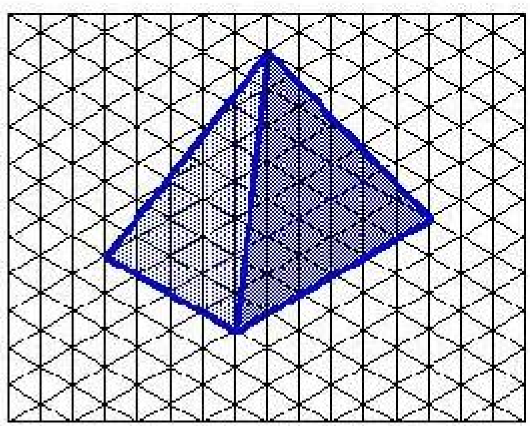


2. Image ID: 6454 Distance: 35980.383

3. Image ID: 5132 Distance: 36270.492

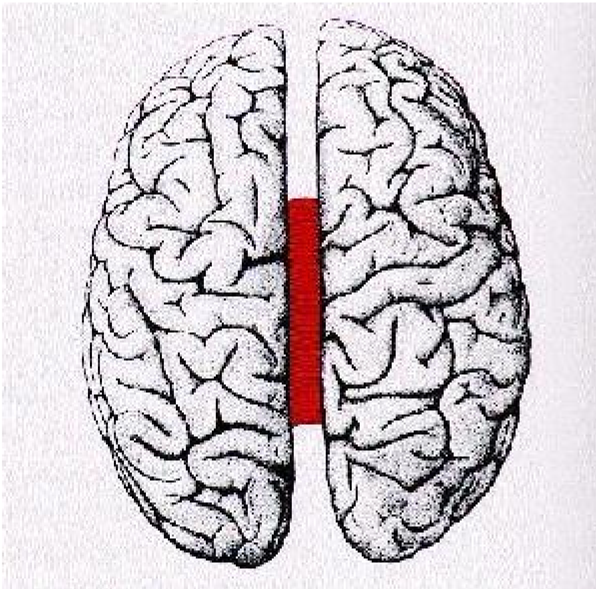


4. Image ID: 6989 Distance: 38271.375



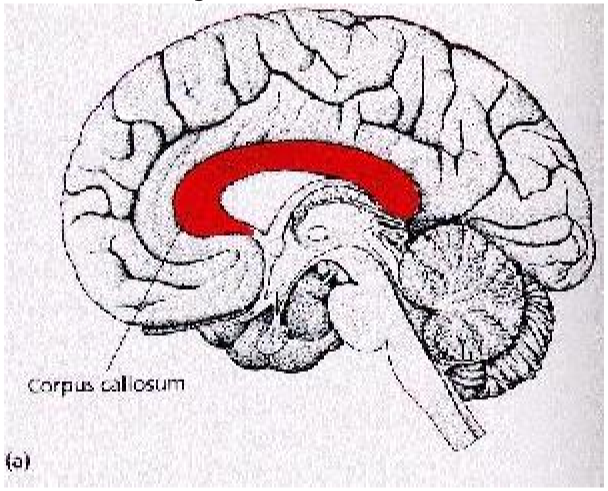
5. Image ID: 8209 Distance: 38983.95

6. Image ID: 3155 Distance: 39237.914



7. Image ID: 7497 Distance: 39320.727


8. Image ID: 3153 Distance: 39748.87




9. Image ID: 8536 Distance: 39790.188

10. Image ID: 5137 Distance: 39922.633

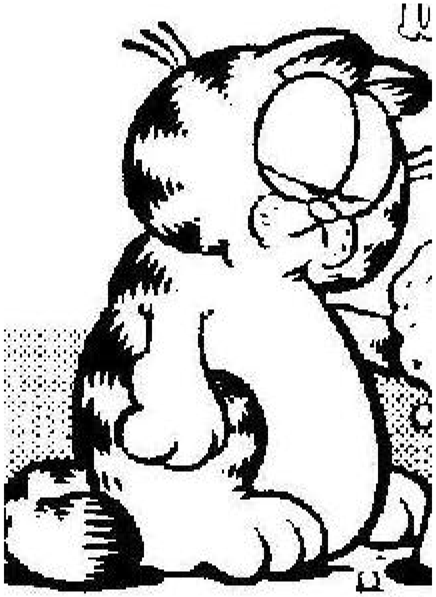
In [ ]:
```
# Convert to grayscale and partition to grid.

from torchvision.transforms import Grayscale
```

```
gs_image = Grayscale()(image300x100)

gs_grid = partition_to_grid(gs_image, num_rows=10, num_cols=10, hor_pixels=30, ver_pixels=10)
```

In [ ]:

```
# Histograms of oriented gradients (HOG).

# Find K most similar images by HOG.

from feature_extractors.HOG import get_hog_vector

hog_vector = get_hog_vector(gs_grid)

# Load hog tensor database into memory.

hog_tensor_dict = tensor_loader('hog_tensors.pt', os.path.abspath(""))

# Correlation similarity: This measures the linear relationship between HOG feature vectors,
# and performs well with features that have a strong linear correlation or dependency in
# their gradient orientation distributions, taking into account directionality.

from scipy.spatial.distance import correlation

print("Similar images by Histogram of Oriented Gradients (HOG) using distance: Correlation")

top_k_by_hog = top_k_ranker(K, hog_vector, hog_tensor_dict, correlation)


for i in range(len(top_k_by_hog)):

    id = top_k_by_hog[i][1]
    distance = top_k_by_hog[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])
```

```
Similar images by Histogram of Oriented Gradients (HOG) using distance: Correlation
1.      Image ID:  5122                 Distance:  0.0
```



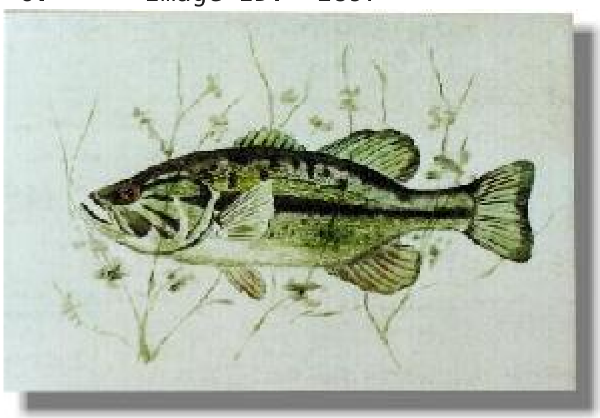
2. Image ID: 3777 Distance: 0.1684781403242568

3. Image ID: 7686 Distance: 0.1724295083705072

4. Image ID: 7281 Distance: 0.17562480133592773

5. Image ID: 5899 Distance: 0.17570036623356555

6. Image ID: 2897 Distance: 0.1812149767199902



7. Image ID: 3300 Distance: 0.18197529353446296

8. Image ID: 6057 Distance: 0.18380253699033222


9. Image ID: 6035 Distance: 0.18473298327785848

```
10.     Image ID:  8150                 Distance:  0.18640012403866713
```

In [ ]:
```
# Resize original image to 224x224 and convert to float tensor.

from torchvision.transforms import transforms

image224x224 = image.resize((224, 224))
```

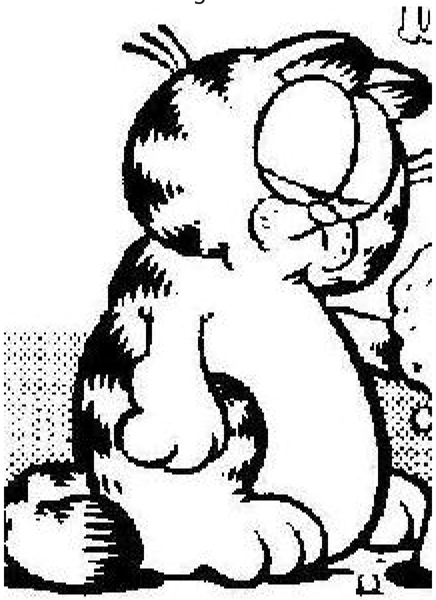
```python
img_tensor = transforms.Compose([transforms.ToTensor()])(image224x224)
```

In [ ]:
```python
# ResNet pre-processing.

# Find K most similar images by each of the 3 layers chosen from ResNet-50.

from feature_extractors.resnet_50 import get_resnet50_feature_vectors

avgpool_vector, layer3_vector, fc1000_vector = get_resnet50_feature_vectors(img_tensor)

# Load each tensor database into memory

avgpool_tensor_dict = tensor_loader('avgpool_tensors.pt', os.path.abspath(""))
layer3_tensor_dict = tensor_loader('layer3_tensors.pt', os.path.abspath(""))
fc1000_tensor_dict = tensor_loader('fc1000_tensors.pt', os.path.abspath(""))
```

In [ ]:
```python
# ResNet-AvgPool-1024.

# Find K most similar images by ResNet-AvgPool-1024.

# Cosine similarity: We use Cosine since the output of the ResNet layer is
# normalized, reducing the importance of magnitude. It is good at discerning
# semantic or structural similarity rather than their absolute feature values.
# It also works well with sparse feature spaces such as those found in CNNs.

from scipy.spatial.distance import cosine

print("Similar images by ResNet-AvgPool-1024 using distance: Cosine")

top_k_by_avgpool = top_k_ranker(K, avgpool_vector, avgpool_tensor_dict, cosine)


for i in range(len(top_k_by_avgpool)):

    id = top_k_by_avgpool[i][1]
    distance = top_k_by_avgpool[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])
```

Similar images by ResNet-AvgPool-1024 using distance: Cosine

1. Image ID: 5122 Distance: 0



2. Image ID: 5137 Distance: 0.18765169382095337

3. Image ID: 5112 Distance: 0.21680545806884766

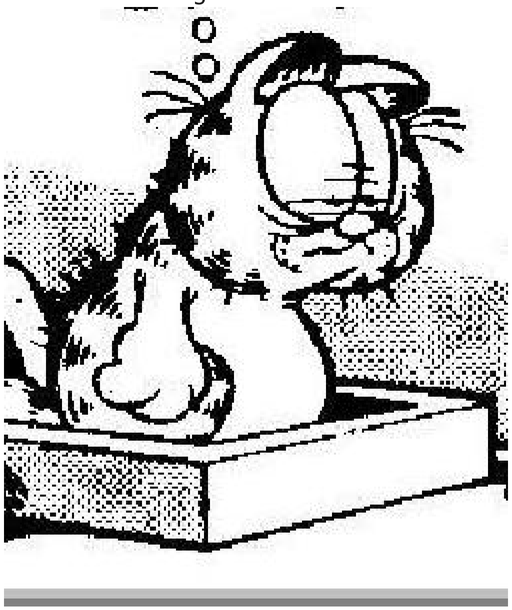
4. Image ID: 7474 Distance: 0.22793978452682495


5. Image ID: 7471 Distance: 0.2319655418395996

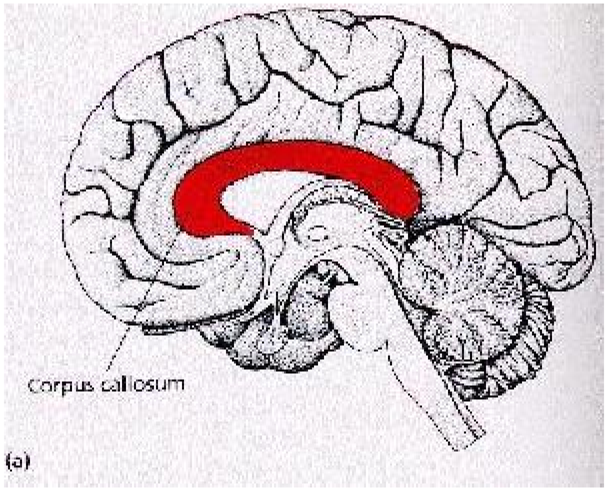
6. Image ID: 2888 Distance: 0.2446938157081604

7. Image ID: 7479 Distance: 0.2543797492980957

8. Image ID: 5132 Distance: 0.25515806674957275



9. Image ID: 3153 Distance: 0.25874292850494385

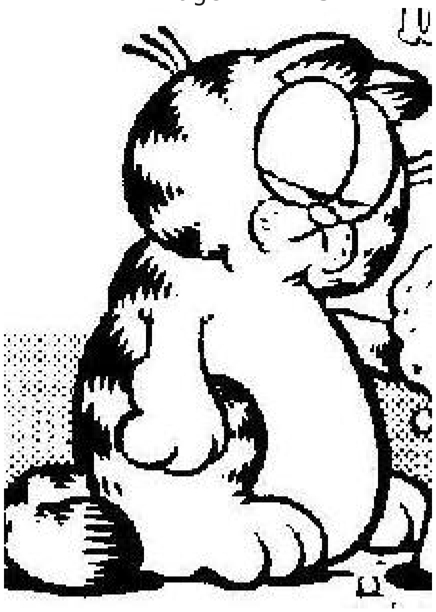



```
10.      Image ID:  7497                    Distance:  0.270865261554718
```

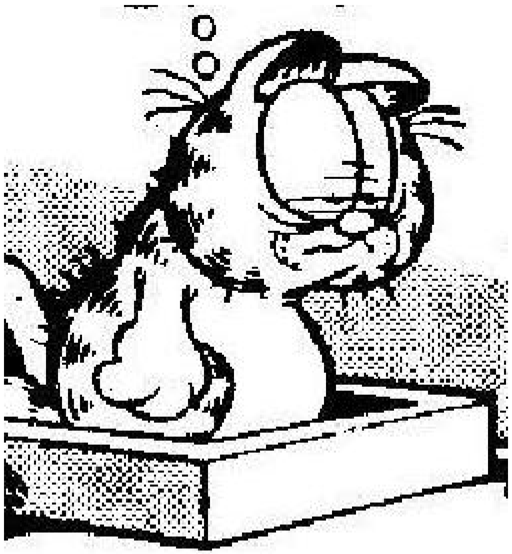

```
In [ ]: # ResNet-Layer3-1024.

        # Find K most similar images by ResNet-Layer3-1024.

        # Cosine similarity: We use Cosine since the output of the ResNet layer is
        # normalized, reducing the importance of magnitude. It is good at discerning
        # semantic or structural similarity rather than their absolute feature values.
        # It also works well with sparse feature spaces such as those found in CNNs.
```

```
print("Similar images by ResNet-Layer3-1024 using distance: Cosine")

top_k_by_layer3 = top_k_ranker(K, layer3_vector, layer3_tensor_dict, cosine)


for i in range(len(top_k_by_layer3)):

    id = top_k_by_layer3[i][1]
    distance = top_k_by_layer3[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])
```

```
Similar images by ResNet-Layer3-1024 using distance: Cosine
1.      Image ID:  5122                 Distance:  0
```

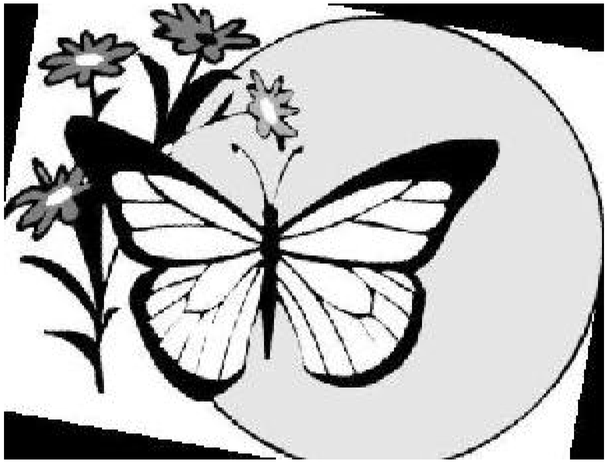


```
2.      Image ID:  5132                 Distance:  0.03839331865310669
```



3. Image ID: 5137 Distance: 0.05274897813796997

4. Image ID: 3383 Distance: 0.05594545602798462



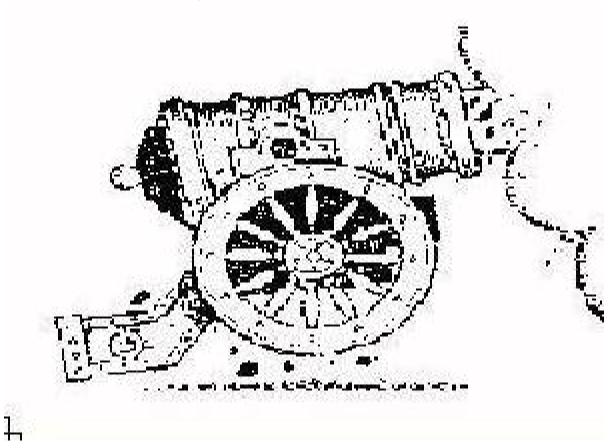
5. Image ID: 2888 Distance: 0.05623418092727661

6. Image ID: 7497 Distance: 0.05633091926574707


7. Image ID: 5112 Distance: 0.059560179710388184

8. Image ID: 7471 Distance: 0.06032222509384155

9. Image ID: 3518 Distance: 0.060375869274139404

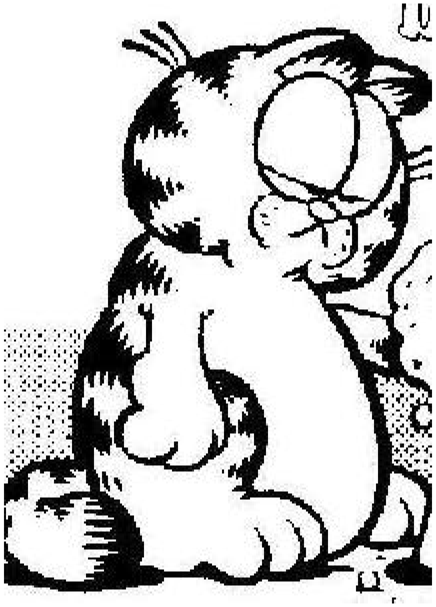


10. Image ID: 7475 Distance: 0.06101977825164795

In [ ]:

```
# ResNet-FC-1000.

# Find K most similar images by ResNet-FC-1000.

# Cosine similarity: We use Cosine since the output of the ResNet layer is
# normalized, reducing the importance of magnitude. It is good at discerning
# semantic or structural similarity rather than their absolute feature values.
# It also works well with sparse feature spaces such as those found in CNNs.

print("Similar images by ResNet-FC-1000 using distance: Cosine")

top_k_by_fc1000 = top_k_ranker(K, fc1000_vector, fc1000_tensor_dict, cosine)


for i in range(len(top_k_by_fc1000)):

    id = top_k_by_fc1000[i][1]
    distance = top_k_by_fc1000[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])
```

```
Similar images by ResNet-FC-1000 using distance: Cosine
1.      Image ID:  5122                 Distance:  0
```



2. Image ID: 5137 Distance: 0.16487550735473633

3. Image ID: 7474 Distance: 0.19566476345062256

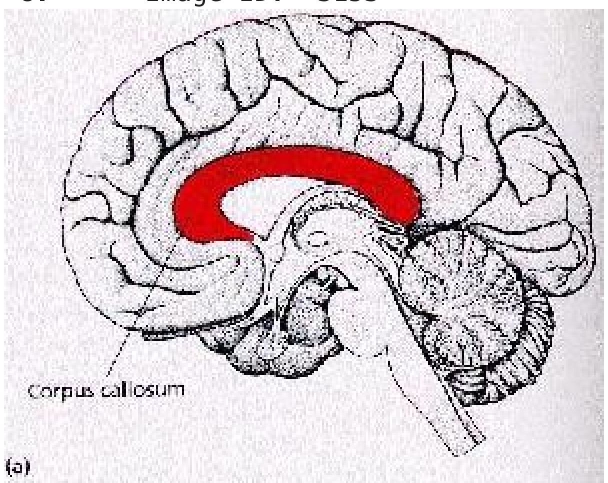

4. Image ID: 7471 Distance: 0.20404183864593506

5. Image ID: 5112 Distance: 0.20469021797180176

6. Image ID: 2888 Distance: 0.2152014970779419

7. Image ID: 7475 Distance: 0.23861569166183472

8. Image ID: 3153 Distance: 0.242974460124969448




9. Image ID: 7497 Distance: 0.24322742223739624


10. Image ID: 5136 Distance: 0.25195860862731934